

Relational Databases

A relational database stores data in structured tables that relate to each other.

Imagine organized spreadsheets where rows can reference rows in other sheets.

Key characteristics

1. Fixed structure (Schema)

Before inserting data, you define the structure.

Example:

Users must have:

id

name

email

You cannot randomly add fields.

2. Relationships are first-class

Tables are designed to connect.

Examples:

User → Orders

User → Posts

Post → Comments

The database **enforces these connections**.

3. Strong data integrity

Relational DBs prevent bad data.

Example:

If a user_id doesn't exist in users, the DB can reject it.

This is called **referential integrity**.

4. ACID guarantees

Relational databases focus on **reliability and correctness**.

ACID means:

- **Atomicity** – operations complete fully or not at all
- **Consistency** – rules are always enforced
- **Isolation** – concurrent operations don't conflict
- **Durability** – data stays even after crashes

Examples of relational databases

Popular ones you'll see everywhere:

- PostgreSQL
- MySQL
- SQLite
- Microsoft SQL Server

Most **serious backend systems use PostgreSQL** today.

Quick Comparison

Feature	Relational DB	NoSQL DB
Structure	Fixed schema	Flexible schema
Data model	Tables	Documents / Key-value
Relationships	Strong	Usually weaker
Consistency	Strong (ACID)	Sometimes eventual
Best for	Structured apps	Massive scale / flexible data

Q. When you should choose SQL vs NoSQL in real backend architecture.

Relational databases (SQL) like PostgreSQL or MySQL store data in structured tables with predefined schemas and strong relationships between tables.

We choose SQL databases when:

- Data has clear relationships (users, orders, payments, etc.)
- Data consistency and integrity are critical
- We need ACID transactions
- The schema is relatively stable

Typical use cases include banking systems, SaaS applications, e-commerce platforms, and authentication systems.

NoSQL databases like MongoDB store data in flexible formats such as documents, key-value pairs, or graphs.

We choose NoSQL databases when:

- Data structure is flexible or frequently changing
- We need high horizontal scalability
- The system handles large volumes of unstructured or semi-structured data
- Relationships between data are not very complex

Typical use cases include real-time analytics, logging systems, social media feeds, and IoT data platforms.

Short summary line (great for interviews):

Relational databases like PostgreSQL are used when the data has strong relationships, requires a fixed schema, and needs ACID transactions for consistency—like in financial systems or SaaS platforms.

NoSQL databases like MongoDB are preferred when the data is semi-structured or unstructured, the schema changes frequently, and the system needs high horizontal scalability.

In modern architectures, teams often use both depending on the specific use case.